Tema 2

Introducción a la orientación a objetos

Programación

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Clases. Atributos, métodos y visibilidad 5
2.1. Definición de la clase 5
2.2. Definición de atributos 6
2.3. Definición de métodos 6
2.4 Definición de visibilidad 9
3. Objetos. Estado, comportamiento e identidad. Mensajes 11
3.1 Definición de objeto 11
3.2 Definición de estado y comportamiento 12
3.3 Definición de Identidad 13
3.4 Definición de un mensaje 13
4. Encapsulado. Visibilidad 15
4.1 Definición de encapsulación 15
4.2 Definición de visibilidad 16
5. Relaciones entre clases 19
5.1. Definición de herencia 19
5.2. Herencia de múltiples fuentes o herencia múltiple 23
6. Principios básicos de la orientación a objetos 26
7. Resumen 30
Glosario 31
Bibliografía 32

Objetivos de la unidad

- Familiarizarnos con los conceptos básicos y fundamentales de la POO. Estos conceptos se centran en la organización de la programación en torno a objetos, en lugar de funciones y procedimientos.
- Comprender los principios de la POO como la encapsulación, la herencia y el polimorfismo.
- Aprender la sintaxis y las características de un lenguaje orientado a objetos
- Identificar y diseñar clases y objetos
  Entender la relación entre clases y objetos

Tech Hub <FP> euroformac

1. Introducción

En esta unidad, exploraremos uno de los paradigmas de programación más importantes y utilizados en la industria del desarrollo de software: la **programación orientada a objetos** (**POO**). La **POO** nos permite organizar nuestro código de una manera más estructurada y modular, lo que nos brinda beneficios significativos en términos de reutilización, mantenibilidad y escalabilidad.

Conoceremos los conceptos fundamentales de la POO y cómo se aplican en la resolución de problemas. Aprenderemos cómo pensar en términos de objetos, atributos y comportamientos, y cómo diseñar clases que representen entidades del mundo real o del problema que estamos abordando.

También nos sumergiremos en los pilares de la POO: **encapsulación**, **herencia** y **polimorfismo**. Estos conceptos nos permitirán escribir código más modular y flexible, y nos brindarán la capacidad de extender y personalizar nuestras clases de manera eficiente.

A lo largo de la unidad, trabajaremos con un lenguaje de programación orientada a objetos y aprenderemos su sintaxis y características específicas. Practicaremos la creación de clases, la creación de objetos y cómo interactúan entre sí.

Tech Hub <FP> euroformac

2. Clases. Atributos, métodos y visibilidad

2.1. Definición de la clase

En pocas palabras, una **clase** es una abstracción que creamos a partir de nuestra experiencia sensible. Los seres humanos tienen una tendencia a **clasificar los objetos** en grupos llamados "clases" en función de sus rasgos compartidos. Como resultado, a pesar de que hay numerosos recipientes diferentes, podemos reconocer un vaso tan pronto como lo vemos, aunque en realidad nunca hayamos visto ese modelo de vidrio específico.

Una clase también se puede definir como la **plantilla** o el **molde** que usamos para crear los objetos. Cada entidad en el mundo de la programación se adscribe a una clase específica. Un objeto surgido a partir de una clase es considerado una representación concreta de dicha clase. Las diferentes clases establecen distintas relaciones de herencia entre sí: una clase puede originarse a partir de otra, en ese caso la clase resultante o clase descendiente hereda los atributos y funciones de la clase de la que deriva o clase progenitora. En Python, todas las clases tienen como primer antecesor una clase en común: la clase **Object**.

Cada clase tiene un código correspondiente (definición de clase) que controla:

- Las características de los objetos de clase.
- Los métodos que pueden usar los objetos de la clase, así como los detalles de cómo lo hacen.

Escribir **código para clases de objetos** es lo que implica la programación orientada a objetos.

Vamos a continuación a profundizar en todos estos conceptos y a explicar su sintaxis en **Python**.

```
class MiClase:
  def __init__(self, atributo1, atributo2):
    self.atributo1 = atributo1
    self.atributo2 = atributo2

  def metodo1(self):
    # código del método 1
    pass
```

```
def metodo2(self):
    # código del método 2
    Pass
```

En este ejemplo, la clase se llama "MiClase". Tiene dos atributos, "atributo1" y "atributo2", que se definen en el método especial __init__, también conocido como constructor de la clase. Luego, la clase tiene dos métodos, "metodo1" y "metodo2", que pueden contener código para realizar ciertas acciones.

Recordemos que la **indentación** es importante en Python, ya que define el alcance de los bloques de código. En el caso de una clase, los atributos y métodos deben estar indentados por cuatro espacios dentro del cuerpo de la clase para indicar que pertenecen a la misma.

2.2. Definición de atributos

Los **atributos** son las características unívocas que permiten distinguir los objetos de una clase u otra. Describen las características o el estado de un objeto y pueden ser accedidos y modificados a través del código del programa. Pueden contener diferentes tipos de datos, como números, texto, booleanos, entre otros. Los atributos permiten representar y almacenar datos relevantes para el funcionamiento y la interacción de los objetos en un programa. También pueden tener diferentes niveles de visibilidad, lo que determina si pueden ser accedidos desde otros objetos o clases. Los atributos son fundamentales en la programación orientada a objetos para modelar y manipular la información de manera estructurada.

2.3. Definición de métodos

Las operaciones o funciones que especifican cómo se comportan los objetos pertenecientes a una clase se denominan **métodos**. La palabra clave **def** las define como funciones.

El parámetro **self** en los métodos de una clase en Python es una referencia al objeto desde el cual se llama el método. Permite acceder a los atributos y métodos de la clase dentro de su propia definición.

Veamos un ejemplo sencillo que muestra cómo se utiliza **self** para acceder a los atributos y métodos de una clase en su propia definición:

```
class MiClase:
    def __init__(self, atributo1, atributo2):
        self.atributo1 = atributo1
        self.atributo2 = atributo2

    def metodo1(self):
        # Acceso al atributo
        print(self.atributo1)

        # Llamada a otro método
        self.metodo2()

    def metodo2(self):
        print("Este es el método 2")

# Crear una instancia de la clase
objeto = MiClase("valor1", "valor2")

# Llamar a un método de la instancia
objeto.metodo1()
```

En este ejemplo, **self.atributo1** y **self.atributo2** se utilizan para acceder a los atributos de la clase MiClase dentro del método metodo1. También se utiliza self.metodo2() para llamar al método metodo2 dentro de la propia clase.

Otro ejemplo sencillo de Clase en Python donde se define un **atributo y un método**:

```
class Saludo:
...    mensaje = "Bienvenido "# Definición de un
atributo
...    def saludar(self, nombre):# Definición de un
método
...    print(self.mensaje + nombre)
...    return
>>> s = Saludo()
>>> s.saludar('Alumno')
```

Bienvenido Alumno. # Salida por pantalla

A continuación, vamos a poner un ejemplo de clase llamada **Coche** en Python para ver cómo se comporta la interacción con los métodos y atributos de la clase.

class Coche:

```
def __init__(self, marca, color, matrícula, modelo,
cuenta_kilometros, velocidad):
    # Propiedades
    self.numero_llantas = 4  # es una constante, siempre
habrá 4 llantas.
    self.marca = marca
    self.color = color
    self.matrícula = matrícula
    self.modelo = modelo
    self.cuenta_kilometros = 0
    self.velocidad = 0
```

El código crea una clase llamada Coche en la cual definimos los atributos número llantas, marca, color, matrícula y modelo. El atributo número_llantas en una constante que siempre valdrá 4.

A continuación, definiremos los métodos de la clase que definirán el comportamiento que tendrán los objetos instanciados de esa clase.

```
def acelerar(self):
      print(f"Soy {self.marca}, estoy acelerando!!")
  def frenar(self):
      print(f"Soy {self.marca}, estoy frenando!!")
  def girar(self, direccion):
      print(f"Soy {self.marca}, estoy girando a la
{direccion}")
  def pitar(self):
      print('Bip Bip Bip')
  def consultar_velocimetro(self):
      return self.velocidad
  def consultar_cuenta_kilometros(self):
      return self.cuenta_kilometros
```

2.4 Definición de visibilidad

Definición

En Python, la visibilidad se refiere a la accesibilidad de variables, atributos y métodos desde diferentes partes del código.

Hay **tres niveles de visibilidad** en Python:

- **Pública (public):** Los miembros públicos son accesibles desde cualquier parte del código, tanto desde dentro de la clase como desde fuera de ella. Se puede acceder a ellos directamente utilizando el nombre del objeto o instancia de la clase.

Ejemplo:

```
class MiClase:
    def __init__(self):
        self.atributo_publico = 10

    def metodo_publico(self):
        print("Este es un método público")

objeto = MiClase()
print(objeto.atributo_publico)  # Acceso a un atributo
público
objeto.metodo_publico()  # Llamada a un método público
```

- **Protegida (protected):** Los miembros protegidos son accesibles desde la propia clase y también desde las subclases. Sin embargo, no se recomienda acceder a ellos directamente desde fuera de la clase o de sus subclases. Para indicar un atributo o método protegido en Python, se utiliza una convención de nomenclatura con un solo guion bajo al comienzo del nombre.

Ejemplo:

```
class MiClase:
    def __init__(self):
        self._atributo_protegido = 20

    def _metodo_protegido(self):
        print("Este es un método protegido")
```

```
objeto = MiClase()
print(objeto._atributo_protegido)  # Acceso a un atributo
protegido (no recomendado)
objeto._metodo_protegido()  # Llamada a un método protegido
(no recomendado)
```

- **Privada (private):** Los miembros privados son accesibles solo desde dentro de la propia clase. No se puede acceder a ellos desde fuera de la clase, incluso si se intenta acceder directamente utilizando el nombre del objeto o instancia de la clase. Para indicar un atributo o método privado en Python, se utiliza una convención de nomenclatura con dos guiones bajos al comienzo del nombre.

Ejemplo:

```
class MiClase:
    def __init__(self):
        self.__atributo_privado = 30

    def __metodo_privado(self):
        print("Este es un método privado")

objeto = MiClase()
# print(objeto.__atributo_privado)  # Error: No se puede
acceder a un atributo privado directamente
# objeto.__metodo_privado()  # Error: No se puede llamar a
un método privado directamente
```

En el caso de los miembros protegidos y privados, aunque no se prohíbe su acceso, es una convención en Python no acceder a ellos directamente desde fuera de la clase, y se recomienda utilizar los métodos públicos de la clase para interactuar con ellos.

Importante

Es importante tener en cuenta que la visibilidad en Python se basa en convenciones y no en restricciones estrictas. A diferencia de otros lenguajes de programación, Python no impide el acceso directo a miembros protegidos o privados, pero se espera que los programadores sigan las convenciones establecidas para escribir código limpio y comprensible.

3. Objetos. Estado, comportamiento e identidad. Mensajes

3.1 Definición de objeto

En Python, un objeto es una instancia de una clase. Una clase es una plantilla que define las propiedades y los comportamientos de un tipo de objeto en particular. Cuando se crea un objeto a partir de una clase, se dice que se ha instanciado la clase, y ese objeto particular tiene sus propios datos y puede realizar acciones según las definidas en la clase.

En resumen, un objeto en Python es una entidad individual que tiene atributos y métodos definidos por su clase correspondiente. Los atributos representan las características o datos asociados con el objeto, mientras que los métodos son las funciones que el objeto puede realizar. Si continuamos con el ejemplo de la **clase Coche** donde habíamos definido la clase como:

```
class Coche:
    def __init__(self, marca, color, matrícula, modelo):
        # Propiedades
        self.numero_llantas = 4 # es una constante, siempre
habrá 4 llantas.
        self.marca = marca
        self.color = color
        self.matrícula = matrícula
        self.modelo = modelo
  def acelerar(self):
        print(f"Soy {self.marca}, estoy acelerando!!")
    def frenar(self):
        print(f"Soy {self.marca}, estoy frenando!!")
    def girar(self, direccion):
        print(f"Soy {self.marca}, estoy girando a la
{direccion}")
    def pitar(self):
        print('Bip Bip Bip')
```

Podemos **realizar la instanciación** de la **clase Coches** para crear los objetos de **tipo Coche**:

Objetos coche, se instancia un nivel de sangría atrás, fuera de la clase.

```
obj_mazda = Coche('Mazda CX5', 'Rojo', 'RED126', '2020')
obj_renault = Coche('Renault Clio', 'Negro', 'FRE009',
'2021')
obj_audi = Coche('Audi A1', 'Blanco', 'DPK312', '2016')
```

Y probar algunos de los atributos que hemos definido en la clase.

Objetos carro, se instancia un nivel de sangría atrás, fuera de la clase.

```
# El Mazda CX5, va a girar a la izquierda
obj_mazda.girar('izquierda')
# El Audi A1, va a girar a la derecha
obj_audi.girar('derecha')
# El Renault Clio va a acelerar
obj_renault.acelerar()
```

3.2 Definición de estado y comportamiento

Un **objeto** es una entidad con identidad bien definida que encapsula estado y comportamiento. Los atributos y relaciones de un estado sirven como representación de ese estado, mientras que las operaciones sirven como representación del comportamiento.

Se debe establecer un propósito claro para un objeto para que se considere una entidad con límites claramente definidos. Los atributos y los métodos son las dos categorías de partes que componen un objeto.

- **El estado en un objeto**

El **estado de un objeto** se define como su entorno actual en un momento determinado. Normalmente, esta condición evoluciona con el tiempo.

Un conjunto de propiedades conocidas como atributos, junto con sus valores y cualquier conexión que el objeto tenga con otros objetos, constituyen el estado de un objeto.

El conjunto completo de atributos del objeto y enlaces a otros objetos, no solo el atributo "estado" o un subconjunto de ellos, define el estado.

- **El comportamiento de un objeto**

El **comportamiento de un objeto** es su segunda cualidad. Las cosas, incluido su comportamiento, deben reflejar las ideas que representan. El comportamiento de un objeto determina cómo se comporta y responde a las solicitudes de otras personas. Las operaciones de un objeto sirven como una representación de su comportamiento.

3.3 Definición de Identidad

La **identidad** se refiere a una propiedad de un objeto o entidad que lo distingue y lo hace único. En el mundo real, dos personas pueden tener los mismos rasgos como un nombre, una fecha de nacimiento o un trabajo, pero no hay duda de que son personas diferentes con sus propias identidades.

Los objetos se pueden pensar de la misma manera. Dos objetos pueden tener el mismo estado (atributos y relaciones), pero aún existen como objetos separados con sus propias identidades.

3.4 Definición de un mensaje

Un mensaje se refiere a una solicitud o instrucción enviada a un objeto para que realice una acción o responda de alguna manera específica. Es una forma de comunicación entre objetos en un sistema de programación orientada a objetos.

Los objetos pueden comunicarse entre sí utilizando mensajes como medio de comunicación. Un mensaje activa la funcionalidad de un objeto receptor. Cuando se completa esta operación, el flujo de control se devuelve al objeto que llama.

Los conceptos aprendidos en este apartado se pueden resumir en el ejemplo a continuación con **Código Python**:

class Coche:

def __init__(self, marca, color, matrícula, modelo)

- Definen el **estado** de un objeto:

```
self.numero_llantas = 4  # es una constante, siempre
habrá 4 llantas.
self.marca = marca
self.color = color
self.matrícula = matrícula
self.modelo = modelo
```

- Definen el **comportamiento** de un objeto:

```
def acelerar(self):
    print(f"Soy {self.marca}, estoy acelerando!!")
```

```
    def frenar(self):
        print(f"Soy {self.marca}, estoy frenando!!")


    def girar(self, direccion):
        print(f"Soy {self.marca}, estoy girando a la
{direccion}")


    def pitar(self):
        print('Bip Bip Bip')
```

La identidad de un objeto viene determinada por el valor de sus atributos o estado:

```
obj_renault = Coche('Renault Clio', 'Negro', 'FRE009',
'2021')
```

4. Encapsulado. Visibilidad

4.1 Definición de encapsulación

De acuerdo con este último principio del paradigma de la programación orientada a objetos, los métodos de acceso deben usarse para cambiar y acceder a los atributos de otras clases en lugar de permitir que una clase modifique directamente esos atributos.

Los **atributos de la clase pueden presentar tres niveles diferentes de privacidad**, que debemos comprender.

- **Público**: Cuando todas las demás clases pueden acceder a un método o atributo, se considera que tiene visibilidad pública. Se hace referencia a una clase o subclase diferente.
- **Privado**: Solo el código de clase en sí puede verlos y acceder a ellos, haciéndolos privados.
- **Protegido**: Solo el código de su clase o una de sus subclases (clases heredadas) puede acceder a este recurso.

Definición

Encapsulación se define como la capacidad de un objeto para ocultar los detalles de implementación de sus operaciones al ofrecer una interfaz de acceso claramente definida.

La encapsulación, también denominada **ocultación de información**, permite a los programadores utilizar una interfaz sin necesidad de conocer los detalles de la implementación.

El programador solo confía en la interfaz gracias a la encapsulación, que elimina la dependencia directa de los detalles de implementación. Mientras no se cambie la interfaz, es posible cambiar la implementación sin tener ningún impacto en los programadores.

Los programadores no se ven afectados por los cambios de implementación, lo que reduce el acoplamiento y evita que cualquier corrección o cambio en una operación obligue a los programadores a realizar cambios. Como resultado, el mantenimiento del sistema se simplifica.

La encapsulación brinda dos formas de protección: resguarda el estado interno de un objeto de la manipulación directa por parte de los clientes y salvaguarda a los programadores de los posibles cambios en los detalles

operativos de la implementación del objeto. La interfaz de los mensajes que un objeto puede recibir constituye el componente esencial de la encapsulación. Toda comunicación con el objeto debe seguir un conjunto predeterminado de operaciones, que está asegurada por la interfaz. Solo las operaciones del objeto tienen acceso a los datos internos del objeto.

Como resultado, es esencial declarar los atributos de un objeto como privados y sus operaciones de interés del programador como públicos.

4.2 Definición de visibilidad

El concepto de visibilidad se refiere a algo que se ve, que es visible, que no se oculta. En el ámbito de la programación, se hace referencia a la visibilidad del código escrito por distintos programadores en cualquier momento y lugar. Esto ocurre cuando se desarrolla un programa, se concluye su desarrollo, se verifica su funcionamiento y se comercializa. Una vez que esto sucede, se puede optar por dejar el código abierto, permitiendo que otros programadores agreguen nuevas funcionalidades, realicen optimizaciones y contribuyan al mantenimiento del programa, adaptándolo a nuevas versiones, mejorando su rendimiento, entre otras acciones.

Esta práctica resulta altamente beneficiosa, ya que, como desarrolladores individuales, no siempre podemos mantener un programa en óptimas condiciones si es extenso, a veces incluso de considerable magnitud, con miles y miles de líneas de código.

Pero ocasionalmente, el programador original es consciente de ciertas áreas de su código que considera "sensibles", "delicadas" o "problemáticas". La mayoría de las veces es el código que él cree que no debe cambiarse porque, hacerlo, cambiaría significativamente la funcionalidad original del programa e incluso podría hacer que dejara de funcionar. Una parte importante del trabajo (y de las intenciones) del programador original se destruiría si otro programador, accidentalmente o a propósito, modifica el código “ sensible o delicado”

En el contexto de situaciones similares a la que se ha descrito en este ejemplo, la mayoría de los lenguajes de programación suelen implementar herramientas de privacidad que resultan valiosas para los programadores. Estas herramientas tienen como finalidad evitar que otro programador acceda, modifique o altere accidentalmente este "código sensible", lo que podría derivar en la alteración del programa original y en su posterior diferencia respecto al estado previo.

En Python empleamos los recursos de visibilidad. La **visibilidad** es una herramienta utilizada en la programación orientada a objetos para otorgar acceso a los métodos y atributos de un objeto. Se pueden usar tres palabras reservadas para personalizar esta calidad.

- **Público**: Cuando todas las demás clases pueden acceder a un método o atributo, se considera que tiene visibilidad pública. Se hace referencia a una clase o subclase diferente.
- **Privado**: Solo el código de clase en sí puede verlos y acceder a ellos, haciéndolos privados.
- **Protegido**: Solo el código de su clase o una de sus subclases (clases heredadas) puede acceder a este recurso.

Los **atributos de clase** pueden presentar **tres niveles diferentes de privacidad**, que debemos comprender.

- **Público:** Todos los atributos que hemos definido hasta el momento tienen un nivel de público. Esto implica que es posible acceder y modificar cualquier atributo de una clase simplemente creando una instancia de dicha clase y utilizando la sintaxis de punto correspondiente.
  Ejemplo:

```
class Vehiculo:

    def __init__(self, marca, color, placa, modelo):
   # Los atributos son públicos
      self.marca = marca
      self.color = color
      self.placa = placa
      self.modelo = modelo
```

- **Protegido**: este concepto está relacionado con la herencia, existe la noción de protección, que establece que un atributo solo puede ser accedido y modificado por la propia clase y sus clases derivadas, en caso de que existan. Es necesario que todos los atributos de la clase Vehículo estén en este estado de protección. Para declarar un atributo como protegido, se requiere utilizar una convención de nomenclatura consistente en emplear un guion bajo (_) como prefijo.
  Ejemplo:

```
class Vehiculo:
```

```
    def __init__(self, marca, color, placa, modelo):
        # Los atributos son ahora protegidos - un guion
bajo
        self._marca = marca
        self._color = color
        self._placa = placa
        self._modelo = modelo
```

- **Privado**: Si un atributo es privado significa que solo los miembros de la clase en la que se definió pueden acceder a él. En Python, un atributo debe tener "guiones bajos dobles" adjuntos para que sea privado. Ejemplo:

```
class Vehiculo:
def __init__(self, numero_cambios=4, color='Negro',
peso='50gr'):

# Los atributos son ahora privados --  doble guión bajo
     self.__numero_cambios = numero_cambios
     self.__color = color
     self.__peso = peso
```

5. Relaciones entre clases

5.1. Definición de herencia

Al usar la **herencia**, puede crear una clase secundaria que comparta métodos y atributos con una clase principal. Además, una clase secundaria tiene la capacidad de redefinir cualquier método o atributo existente, así como anularlos.

La clase de la que queremos heredar se puede pasar como parámetro para crear una clase secundaria. Usaremos la clase Perro, que se deriva de Animal, en el siguiente ejemplo para demostrar cómo se puede usar la herencia en Python. Tan sencillo:

- Definimos una **clase padre:**

```
class Animal:
    pass
```

- Creamos **una clase hija que hereda de la clase padre:**

```
class Perro(Animal):
    pass
```

- Podemos comprobar como efectivamente la clase Perro es la hija de Animal usando **bases:**

```
print(Perro.__bases__)
# (<class '__main__.Animal'>,)
```

- De manera similar podemos ver que clases descienden de una en concreto con **subclases**:

```
print(Animal.__subclasses__())
# [<class '__main__.Perro'>]
```

La herencia es un concepto fundamental en la programación orientada a objetos que permite la creación de jerarquías de clases y la reutilización de código. Al heredar de una clase principal (también conocida como clase padre o superclase), una clase secundaria (también conocida como clase hija o subclase) adquiere automáticamente los atributos y métodos de la clase principal. En esta situación, podemos adherirnos a la **filosofía DRY** usando los componentes comunes para crear una **clase Animal** de la cual el resto herede en lugar de definir un número de clases para cada animal.

Importante

Es importante recordar que la herencia debe utilizarse de manera cuidadosa y reflexiva. No todas las similitudes entre clases justifican la herencia, y en algunos casos, otras técnicas, como la composición, pueden ser más apropiadas.

Sabías que

El principio DRY (Don't Repeat Yourself) es un concepto fundamental en el desarrollo de software que promueve la eliminación de la duplicación de código.

El principio **DRY** establece que cada pieza de conocimiento o lógica en un programa debe tener una única representación autoritativa y no redundante.

El objetivo del principio DRY es reducir la repetición innecesaria de código, ya que la duplicación puede conducir a varios problemas, como mayor mantenimiento, aumento de posibles errores y mayor dificultad para realizar cambios o mejoras en el código.

Cuando se aplica el principio DRY, se busca encapsular la lógica común o repetitiva en funciones, métodos, clases, módulos o bibliotecas reutilizables. En lugar de copiar y pegar código similar en diferentes partes del programa, se crea una abstracción o componente reutilizable que se puede invocar en múltiples lugares.

Al seguir el principio DRY se logra un código más limpio, modular y mantenible. Los beneficios incluyen una mayor legibilidad del código, una menor posibilidad de introducir errores y una mayor eficiencia en el desarrollo, ya que los cambios y actualizaciones solo deben realizarse en un lugar.

Además del DRY, otros principios y prácticas, como la modularidad, la reutilización de código y la programación orientada a objetos, también ayudan a reducir la duplicación y promover la eficiencia en el desarrollo de software.

En resumen, el principio DRY es una guía importante en programación que busca evitar la duplicación de código innecesaria. Al aplicar este principio, se promueve la reutilización, la modularidad y el mantenimiento eficiente del código. Sigamos usando el ejemplo de los perros y otros animales. Vamos a crear una clase principal llamada **Animal** que contenga todos los atributos y métodos posibles que un animal podría tener. En programación, es crucial encontrar el denominador común. Examinemos los atributos:

Como todos los animales pertenecen a una especie, tenemos **una especie**.

La **edad** es otro factor porque todos los seres vivos nacen, crecen, se reproducen y eventualmente mueren.

Adicionalmente, existen las siguientes funcionalidades o métodos.

- El método que usamos se llama "**hablar**" y cada animal lo usará de una manera única. Los caballos relinchan, las abejas zumban y los perros ladran.
- Un método para desplazarse o moverse. Algunos animales lo harán mientras caminan, mientras que otros lo harán mientras vuelan.
- Por último, pero no menos importante, una forma común de descripción.

Describimos la clase principal y enumeramos una serie de características que se aplican a todos los animales, como ya hemos mencionado.

```
class Animal:
    def __init__(self, especie, edad):
        self.especie = especie
        self.edad = edad

    # Método genérico pero con implementación
particular
    def hablar(self):
        # Método vacío
        pass
    # Método genérico pero con implementación
particular
    def moverse(self):
        # Método vacío
        pass
    # Método genérico con la misma implementación
    def describeme(self):
        print("Soy un Animal del tipo",
type(self).__name__)
```

Como resultado, ya tenemos la clase genérica **Animal**, que generaliza los rasgos y habilidades que puede poseer cualquier animal. Ahora creamos una clase llamada **Perro** que se deriva de **Animal**. Para demostrar cómo se heredan los métodos y atributos de forma predeterminada, primero crearemos una clase vacía.

```
# Perro hereda de Animal
class Perro(Animal):
    pass
mi_perro = Perro('mamífero', 10)
mi_perro.describeme()
# Soy un Animal del tipo Perro
```

El contenido de la clase principal se transfirió a una nueva clase con solo unas pocas líneas de código. Dado que cada animal se comporta de manera diferente, vamos a crear una serie de animales concretos y reemplazar algunos de los métodos que la clase **Animal** había definido, como hablar o moverse.

Tomando EL ejemplo de la clase Animal y la subclase Abeja, puedes definir la clase Animal con métodos generales como hablar () y moverse (). Luego, al crear la subclase Abeja, puedes reemplazar el método hablar () y moverse () para adaptarse al comportamiento específico de una abeja. Además, podemos agregar un nuevo método llamado picar () que es exclusivo de las abejas.

```
class Perro(Animal):
    def hablar(self):
        print("Guau!")
    def moverse(self):
        print("Caminando con 4 patas")

class Vaca(Animal):
    def hablar(self):
        print("Muuu!")
    def moverse(self):
        print("Caminando con 4 patas")

class Abeja(Animal):
    def hablar(self):
        print("Bzzzz!")
    def moverse(self):
        print("Volando")

    # Nuevo método
    def picar(self):
        print("Picar!")
```

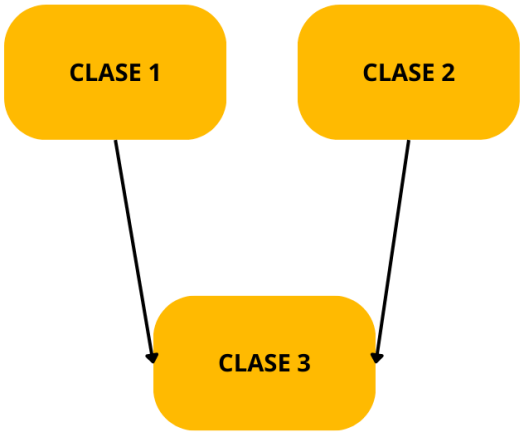
- Esto permite que cada subclase tenga su propio comportamiento único mientras hereda los atributos y otros métodos de la clase base. Cada animal puede comportarse de manera diferente, pero aún se beneficiará de los métodos comunes definidos en la clase Animal de tal manera que tenga la clase abeja los siguientes métodos: Heredados directamente de la **clase padre**: descríbeme ()
- Heredados de la **clase superior (padre)** pero **modificados**: hablar () y moverse ()
- Creados en la **clase hija** por lo tanto no existentes en la clase padre: picar ()

```
#Creación de los objetos
mi_perro = Perro('mamífero', 10)
mi_vaca = Vaca('mamífero', 23)
mi_abeja = Abeja('insecto', 1)

mi_perro.hablar()
mi_vaca.hablar()
# Guau!
# Muuu!

mi_vaca.describeme()
mi_abeja.describeme()
# Soy un Animal del tipo Vaca
# Soy un Animal del tipo Abeja

mi_abeja.picar()
# Picar!
```

5.2. Herencia de múltiples fuentes o herencia múltiple

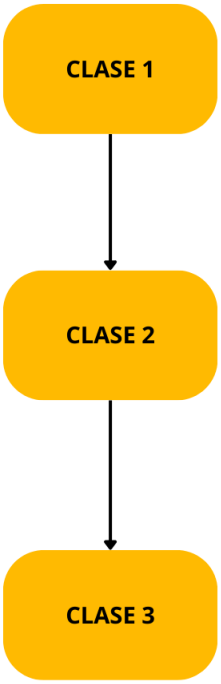
Se pueden realizar múltiples **herencias** en Python. De manera similar a la **herencia simple**, la **herencia múltiple** involucra una clase que deriva sus miembros de varias clases principales diferentes.

Veamos un ejemplo. Tenemos dos clases, Class1 y Class2, por un lado, y Class3, que deriva de las dos primeras, por el otro. Como resultado, se le transmitirán todas las técnicas y características de ambos.




```
class Clase1:
    pass
class Clase2:
    pass
class Clase3(Clase1, Clase2):
    pass
```

También es posible que una clase herede de otra clase, que a su vez puede heredar de la primera.



```
class Clase1:
    pass
class Clase2(Clase1):
    pass
class Clase3(Clase2):
    pass
```

Consultar el **MRO** o **Resolución de orden de método**, es la mejor manera de averiguar qué método se está utilizando. El orden de búsqueda de los métodos está contenido en la **tupla** que devuelve esta función. Como se anticipó, comienza en la clase misma y asciende a la clase principal, de izquierda a derecha.

```
class Clase1:
    pass
class Clase2:
    pass
class Clase3(Clase1, Clase2):
    pass

print(Clase3.__mro__)
# (<class '__main__.Clase3'>, <class '__main__.Clase1'>,
<class '__main__.Clase2'>, <class 'object'>)
```

6. Principios básicos de la orientación a objetos

Hay cuatro principios básicos de la tecnología orientada a objetos:

- **Abstracción**

Definición

La **abstracción** de un objeto en el contexto de la programación orientada a objetos se refiere a la capacidad de representar un objeto de manera simplificada, enfocándose únicamente en los aspectos relevantes y ocultando los detalles internos complejos.

La **abstracción** permite describir un objeto en términos de sus características y comportamientos esenciales, sin preocuparse por cómo se implementa internamente. En lugar de trabajar directamente con los detalles internos de un objeto, nos centramos en su funcionalidad y en cómo interactúa con otros objetos.

La abstracción se logra a través de las clases en la programación orientada a objetos. Una clase proporciona una plantilla o un plano para crear objetos y define las características (atributos) y el comportamiento (métodos) que los objetos de esa clase tendrán.

Al utilizar la abstracción, podemos crear objetos basados en una clase, sin tener que conocer todos los detalles de su implementación interna. Podemos utilizar los atributos y los métodos definidos en la clase para interactuar con el objeto y realizar las operaciones necesarias, sin preocuparnos por cómo se manejan internamente.

La abstracción nos permite separar la estructura y el comportamiento de un objeto de su implementación interna, lo que facilita la reutilización de código, el diseño modular y el desarrollo de sistemas más flexibles y mantenibles.

En resumen, la abstracción de un objeto implica representar un objeto de manera simplificada, centrándose en sus características y comportamientos esenciales, sin preocuparse por los detalles de implementación interna. La abstracción se logra a través de las clases en la programación orientada a objetos y permite separar la estructura y el comportamiento de un objeto de su implementación específica**.** En el ejemplo de un avión, podemos observar que el objeto físico tiene una amplia variedad de características que pueden

ser relevantes en diferentes contextos o aplicaciones. En el caso de una aplicación de gestión de embarque, solo las propiedades relacionadas con los asientos serían de interés, como el número de asientos, su disponibilidad, su ubicación en la aeronave, etc.

Sin embargo, en otras aplicaciones de programación de vuelos, como el seguimiento en tiempo real de aviones o la planificación de rutas, las propiedades como la altitud, la posición del GPS, la dirección, la velocidad, los aeropuertos de origen y destino, o la distancia recorrida pueden ser más relevantes.

La programación de aplicaciones debe considerar cuidadosamente qué propiedades de un objeto son necesarias y relevantes para la funcionalidad específica de la aplicación. Esto implica seleccionar y manejar solo las propiedades del objeto que son importantes para el contexto en el que se está trabajando, y descartar o no prestar atención a las propiedades que no son relevantes para ese contexto en particular.

- **Polimorfismo**

Definición

El **polimorfismo** en Python se refiere a la capacidad de objetos de diferentes clases de comportarse de manera similar o responder a una misma llamada de método, aunque cada objeto pueda tener una implementación específica y diferente de ese método. En otras palabras, el polimorfismo permite que distintos objetos puedan ser tratados de manera uniforme, a pesar de tener estructuras y comportamientos diferentes.

En Python, cuando una clase hereda de otra clase, puede sobrescribir (override) los métodos de la clase base para proporcionar una implementación específica. Sin embargo, es posible que varias clases hijas sobrescriban el mismo método de la clase base de maneras diferentes.

El polimorfismo se utiliza mediante la llamada a métodos en objetos. Cuando se llama a un método en un objeto, Python busca la implementación adecuada del método en la clase del objeto. Si la clase del objeto tiene una implementación propia del método, esa implementación será utilizada. Si la clase no tiene una implementación propia, se buscará en las clases superiores en la jerarquía de herencia hasta encontrar una implementación adecuada.

El polimorfismo permite escribir código más genérico y flexible, ya que se pueden utilizar objetos de diferentes clases en una misma operación sin preocuparse por sus implementaciones específicas. Esto facilita la reutilización de código y promueve un mayor modularidad en los programas.

En resumen, el polimorfismo en Python permite que diferentes objetos puedan ser tratados de manera uniforme, a pesar de tener estructuras y comportamientos diferentes. Se basa en la herencia y la capacidad de sobrescribir métodos en las clases hijas, lo que permite que los objetos respondan a llamadas de métodos de manera específica según su clase, pero en un contexto más general. En la siguiente ilustración se puede observar que, aunque las clases de abajo tengan los mismos nombres en los métodos, la forma de calcular dichos métodos (área y perímetro) van a variar en función del polígono correspondiente.

- **Modularidad**

La capacidad de dividir una aplicación en módulos o partes de la aplicación más pequeños, cada uno de los cuales debe ser lo más independiente posible de los demás, se conoce como modularidad.

Por ejemplo, podemos utilizar funciones o módulos ya hechos dentro de otros programas.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
#
# menu.py
#
# Copyright 2020 pyro <pythones.net>
# Módulo para presentar menus
def menu(a, b): #Creamos la función menu (3
parámetros)
print ("Menu:") #Imprimimos las opciones (parámetros a
y b)
print(a)
print(b)
r = True
while (r == True): #Iniciamos un bucle siempre que "r"
sea true
#Solicitamos una entrada entero
nsel = int(input("Ingresa un número de opción:"))
#Ahora evaluamos esa entrada almacenada en "nsel"
if (nsel == 1): #Condicional = Si indica 1
print ("Gracias!")
return nsel #RETORNAMOS el valor de la variable
```

```
r = False #Rompemos el bucle porque ya eligió
elif (nsel == 2): #Condicional = Si indica 2
print ("gracias!")
return nsel #RETORNAMOS el valor de la variable
r = False #Rompemos el bucle porque eligió
else:
print ("Selecciona una opción válida")
"""Al no elegir o colocar otro número
no rompemos el bucle por lo que vuelve a
solicitar entrada"""
```

Y ahora utilizamos la función menú definida anteriormente dentro del programa principal.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
#
# persona.py
#
# Copyright 2020 pyro <pythones.net>
# Aplicación de ejemplo clases y modularidad
from menu import * #Importamos nuestro módulo menú
(IMPORTANTE NO OLVIDAR)
class persona (object): #Creamos una clase Persona
def __init__(self, nombre): #Constructor y atributos
self.nombre = nombre #Definimos valor del atributo
def elegir (self): #Creamos el método elegir
nsel = 0 #Variable nsel seteada en "0"
print ("Hola", self.nombre, "es momento de elegir")
#Llamamos la función menu y almacenamos su retorno en
"nsel"
nsel = menu("Opción1 = Comprar Periódico","Opción2 =
No comprar")
#Y ahora trabajamos con esa elección
if (nsel == 1):
print("Has elegido la opción 1: Comprar")
else:
print("Has elegido la opcion 2: No comprar nada,
tacaño!")
Pepe = persona("Pepe") #Instancia
Pepe.elegir() #Llamamos el método elegir de la clase
persona
```

De esta manera dividimos el programa en partes más pequeñas que podemos reutilizar en otros programas.

7. Resumen

Los conceptos más relevantes de esta unidad son:

Una clase en Python es una plantilla o un modelo que define la estructura, el comportamiento y los atributos de un objeto. Proporciona una manera de crear objetos que comparten características comunes.

Un objeto en Python es una instancia de una clase. Es una entidad individual con sus propios datos y comportamientos, creada a partir de la clase. Los objetos se utilizan para representar entidades del mundo real o conceptos abstractos.

Los atributos son variables que se definen dentro de una clase y almacenan información específica del objeto. Representan las características o propiedades del objeto.

Los métodos son funciones definidas dentro de una clase que definen el comportamiento del objeto. Los métodos pueden acceder a los atributos de la clase y realizar operaciones sobre ellos.

La herencia de clases es un mecanismo en Python que permite crear una clase nueva basada en una clase ya existente, heredando sus atributos y métodos. La clase nueva se denomina clase hija o subclase, y la clase de la que se heredan los atributos y métodos se llama clase padre o superclase. La herencia permite reutilizar código y establecer relaciones jerárquicas entre las clases.

Encapsulación es un concepto en programación que se refiere a la ocultación de los detalles internos de una clase y la protección de sus atributos y métodos. Permite que los datos y el comportamiento de una clase sean accesibles solo a través de interfaces controladas, lo que ayuda a prevenir modificaciones no autorizadas.

La visibilidad en Python se refiere a qué atributos y métodos de una clase son accesibles desde fuera de la clase.

Glosario

- **Clase:** En programación, una clase es una estructura que define un conjunto de datos y métodos (funciones) relacionados. Proporciona una plantilla para crear objetos que comparten características comunes y comportamientos.
- **Clase hija:** También conocida como subclase o clase derivada, es una clase que hereda atributos y métodos de otra clase llamada clase padre o superclase. La clase hija puede añadir nuevos atributos y métodos o modificar los existentes.
- **Clase padre:** También conocida como superclase o clase base, es una clase de la cual se deriva otra clase llamada clase hija. La clase padre proporciona atributos y métodos que pueden ser heredados y utilizados por las clases hijas.
- **Herencia múltiple:** La herencia múltiple es un concepto en la programación orientada a objetos que permite que una clase herede atributos y métodos de múltiples clases padres. Esto permite que la clase hija tenga características combinadas de varias clases.
- **Indentación:** En Python, la indentación se refiere a la estructura de sangría utilizada para definir bloques de código. Los bloques de código se definen mediante la indentación, generalmente utilizando espacios o tabulaciones. La indentación es importante en Python, ya que determina la estructura y ejecución del código, como en la definición de clases, métodos o bucles.
- **Método:** En programación, un método es una función definida dentro de una clase que define el comportamiento de un objeto de esa clase. Los métodos son llamados para realizar operaciones específicas en un objeto y pueden acceder a los atributos de la clase a través del parámetro especial "self".
- **Mensaje:** En programación orientada a objetos, el término "mensaje" se refiere a la invocación de un método en un objeto. Cuando se llama a un método en un objeto, se envía un mensaje al objeto para que ejecute el código asociado con ese método.
- **Objeto**: En programación orientada a objetos, un objeto es una instancia de una clase. Representa una entidad específica con sus propios datos (atributos) y comportamientos (métodos). Los objetos son las entidades principales con las que se trabaja en la programación orientada a objetos.

Bibliografía

“ **Programación orientada a objetos**” . El Libro De Python.

https://ellibrodepython.com/programacion-orientada-a-objetos.

“ **El tutorial de Python**” . Python documentation.

https://docs.python.org/es/3/tutorial/